Funciones

Las funciones son estructuras en programación que nos permiten la reutilización de código. Todo código que entendemos se puede llegar a repetir podemos separarlo en un bloque y realizar llamadas a este bloque las veces que sea necesario. También el uso de funciones nos permite mejorar la legibilidad del código, de tal manera que en nuestras líneas principales veamos una idea de la ejecución del programa en forma más simplificada. Por ejemplo, desarrollamos un programa que contiene un menú de opciones y este menú de opciones se dibuja de forma gráfica en una ventana o pantalla de consola; entonces este menú lo podemos crear en una función a parte y realizar la llamada a la misma cuando se requiera el uso del menú, de tal manera que en el código principal en vez de contener todas las líneas del menú solo vemos la llamada a la función. Un último caso en el que las funciones nos simplifican la tarea es cuando encontramos un inconveniente o necesitamos realizar un re diseño del codigo, volviendo al caso anterior del menú si vemos por pantalla que existe un error o queremos cambiarle algo basta solamente con trabajar en las líneas de la función.

Estructura de una función en C

```
tipo_dato_devuelto     nombre_de_la_función     (argumentos)
    {
                            cuerpo de la función
    }
```

Toda función que se ejecuta tiene la posibilidad de devolvernos un resultado, el primer elemento de la función es definir el tipo de dato devuelto, también podemos crear funciones que no devuelvan nada. Ejemplos: tenemos una función que calcula un promedio, entonces esta función devuelve un tipo de dato flotante; o tenemos una función que muestra datos de un elemento entonces esta función se dice que devuelve ***void***, ya que es tipo de dato vacío que no devuelve nada.

El nombre de la función puede ser cualquiera que no contenga caracteres especiales ni empiece con números, este nombre es la referencia que utilizamos para realizar la llamada a la función. Si creamos la función que realiza una suma, es probable que a la función le pongamos de nombre “suma”.

Los argumentos de la función son todos los elementos que recibe la función para poder realizar alguna tarea. Cada función puede recibir ninguno, uno o más de un argumento. Los argumentos se cargan con el tipo de dato y un nombre auxiliar a utilizar dentro de la función.

Por último, el cuerpo de la función va entre llaves y contiene todo el código de ejecución de la función.

Si prestamos atención a lo realizado hasta ahora, vemos que nuestro programa principal es también una función, se llama ***main***, devuelve un tipo de dato ***int*** y no recibe nada por argumentos:

```c
int main()
{
    printf("El main es una funcion");
    return 0;
}
```

Funciones de ejemplo

Crearemos un programa que pide dos números y mediante una función calcule la suma:

```
int suma(int,int);

int main()
{
    int x,y;
    int resultado;
    printf("Ingrese dos numeros\n");
    scanf("%d",&x);
    fflush(stdin);
    scanf("%d",&y);
    fflush(stdin);
    resultado=suma(x,y);
    printf("La suma de %d y %d es %d",x,y,resultado);
    return 0;
}

int suma(int a,int b)
{
    return a+b;
}
```

Veamos algunas cuestiones: en primer lugar, la función la nombramos arriba del main, estamos diciendo que la declaramos, pero no la implementamos. Esto quiere decir que el programa principal sabe que existe una función que se llama suma, y que la tiene que buscar abajo cuando la necesite.

La llamada la realizamos utilizando el nombre de la función y como esta devuelve un entero, debemos asignar una variable entera para guardar ese resultado. Otra manera es realizar la llamada a la función directamente desde el printf, evitando la necesidad de crear una variable adicional

```
int suma(int,int);

int main()
{
    int x,y;
    printf("Ingrese dos numeros\n");
    scanf("%d",&x);
    fflush(stdin);
    scanf("%d",&y);
    fflush(stdin);
    printf("La suma de %d y %d es %d",x,y,suma(x,y));
    return 0;
}

int suma(int a,int b)
{
    return a+b;
}
```

Cuando la llamada a la función es realizada, esta recibe dos copias de las variables originales, en este caso en las variables internas a y b se recibe una copia de los valores almacenados en x e y. La función realiza la operación matemática y con la palabra reservada ***return*** devolvemos el resultado.

```c
void menu();

int main()
{
    menu();
    return 0;
}

void menu()
{
    printf("Opcion 1\n");
    printf("Opcion 2\n");
    printf("Opcion 3\n");
    printf("Opcion 4\n");
}
```

En el ejemplo anterior creamos una función que no devuelve nada y tampoco necesita argumentos para trabajar, simplemente imprime un menú. La llamada se realiza solamente nombrando a la función.

Es importante destacar que todas las variables creadas dentro de la función son únicamente accesibles por la función, de tal manera que esta elimina las variables creadas al finalizar la ejecución de la misma.

```c
void intercambio(int a,int b)
{
    int c;
    c=a;
    a=b;
    b=c;
}
```

En este caso la variable entera C únicamente existe dentro del campo de la función.